



Learning to Predict Boolean Satisfiability

A QRefactorBench-Inspired Neural Network Case Study

Generate → inspect → clean → train → evaluate
An auditable workflow for CA6000

Name / Student ID / Partner: complete before submission

3,000 instances

30 features; exact SAT labels.

A real trained MLP

PyTorch weights and history.

Reproducible code

No API key or QPU required.

From a research case to a dataset

Existing classical program

pilot-001: enumerate assignments and return whether all clauses can be satisfied.

Labels come from exhaustive evaluation of the original code.

Independent input instances

4–5 variables; 2-CNF, 3-CNF and mixed formulas.
Random clauses, no planted labels.

3,000 accepted canonical groups;
1,677 UNSAT / 1,323 SAT.

Remove duplicates under literal/clause order and all variable renamings.
Save the generator seed, source hash, formulas and labels.

Coursework data are separate from DRAFT benchmark cases and quantum-suitability labels.

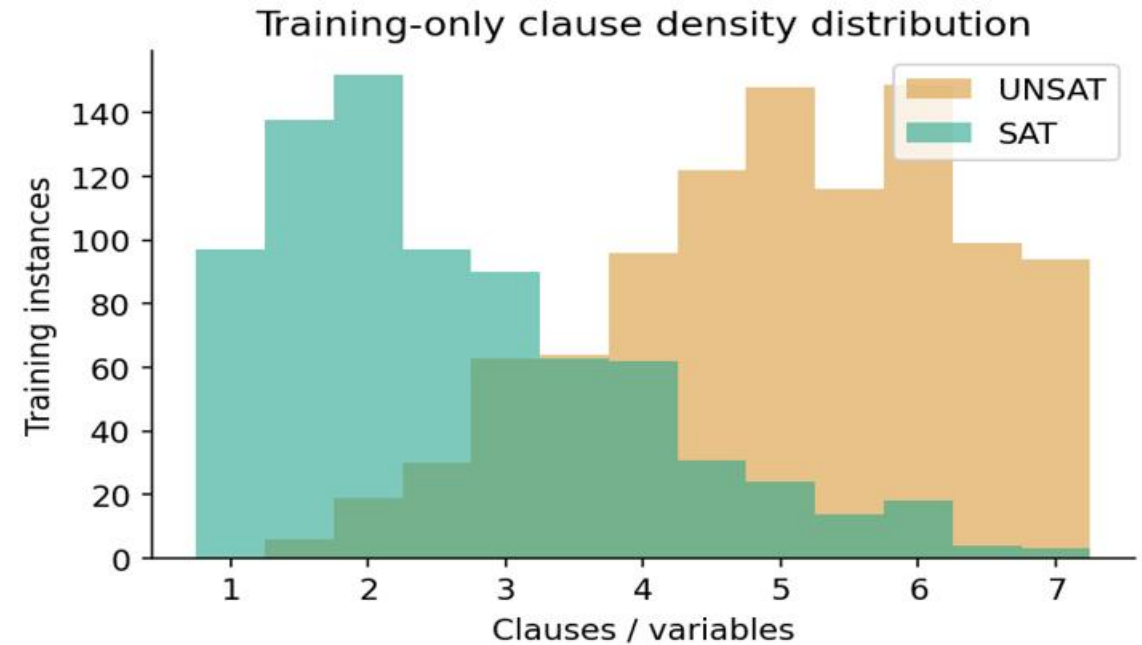
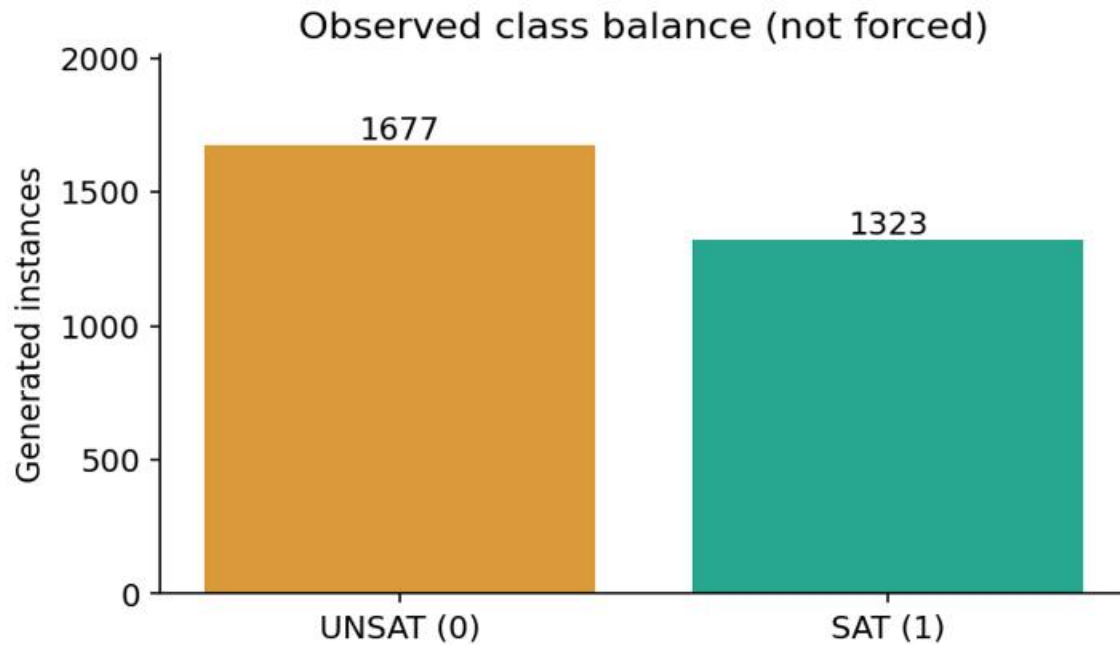
Make data quality visible and reproducible

Issue in training copy	Detected	Pandas repair
Missing feature cells	8	Training-median imputation
Text / infinity / negative counts	3 / 1 / 2	to_numeric + replace + mask
Duplicates / invalid label	4 / 1 rows	drop_duplicates + label filter
Impossible literal total	1: 100,000,000	Domain check → NaN → impute

Errors are intentionally injected only into the training copy.

1,799 clean rows; 15 imputed cells. Raw, validation and test records stay intact.

Explore class balance and formula structure



Cleaned training statistics (all 30 features are available in CSV)

n_clauses	mean=17.718	median=18.000	variance=70.820
clause_variable_ratio	mean=3.902	median=4.000	variance=3.089
total_literals	mean=45.540	median=42.000	variance=555.293

Learn from training data; reserve the test set

Train: 1,799

Fit medians, means and scales.
Learn network parameters.

Validation: 600

Select the checkpoint using
validation BCE loss only.

Test: 601

Evaluate after both models
and threshold are fixed.

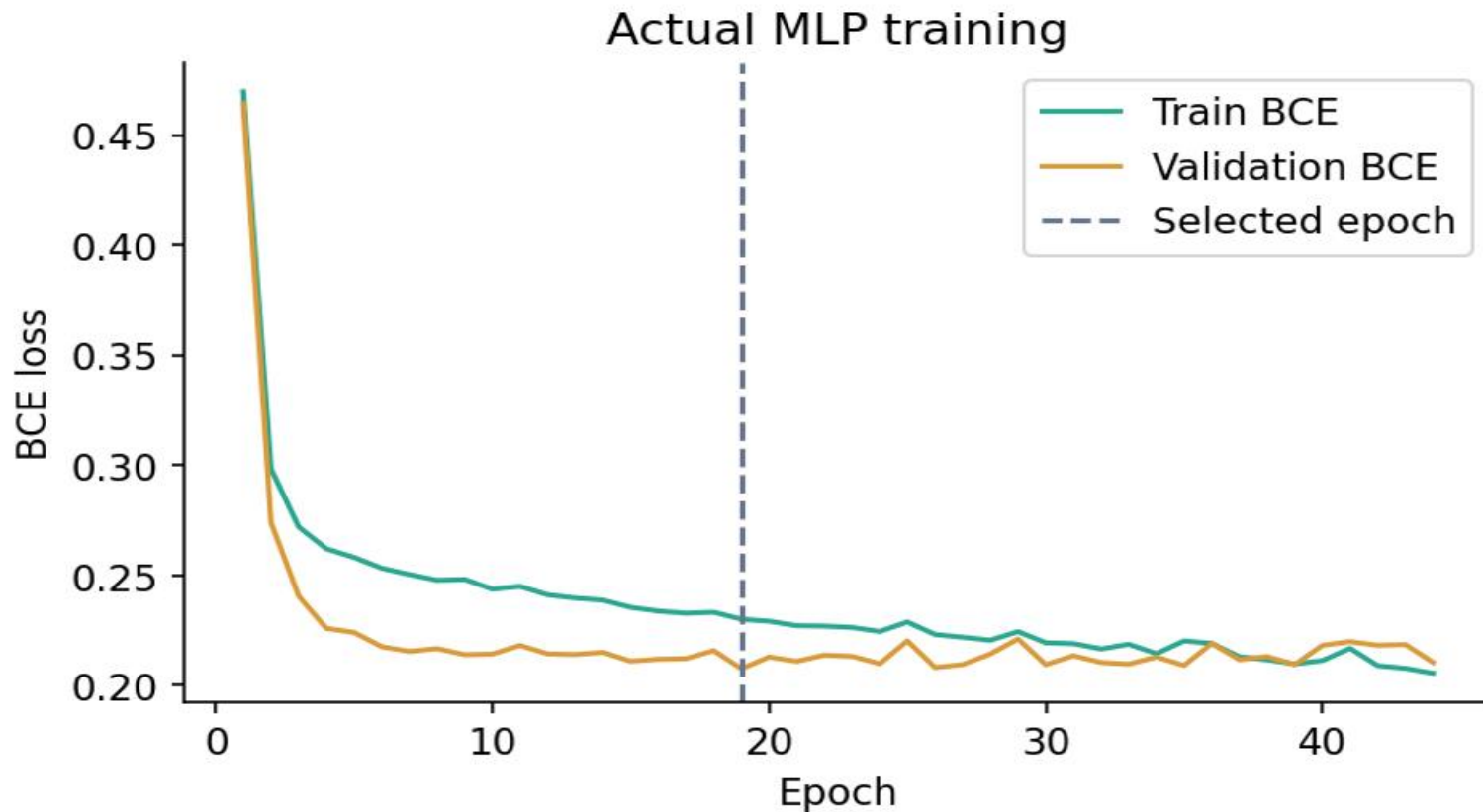
MLP: 30 → 64 ReLU → 32 ReLU → 1 logit

4,097 parameters · BCEWithLogitsLoss · Adam · sigmoid threshold = 0.5

Compare with majority class and a separately trained linear classifier.

Features use formula statistics only — no solver output, labels or runtime.
Canonical groups are disjoint; ID and formula hash are never model inputs.

The network was actually trained locally



Fixed configuration

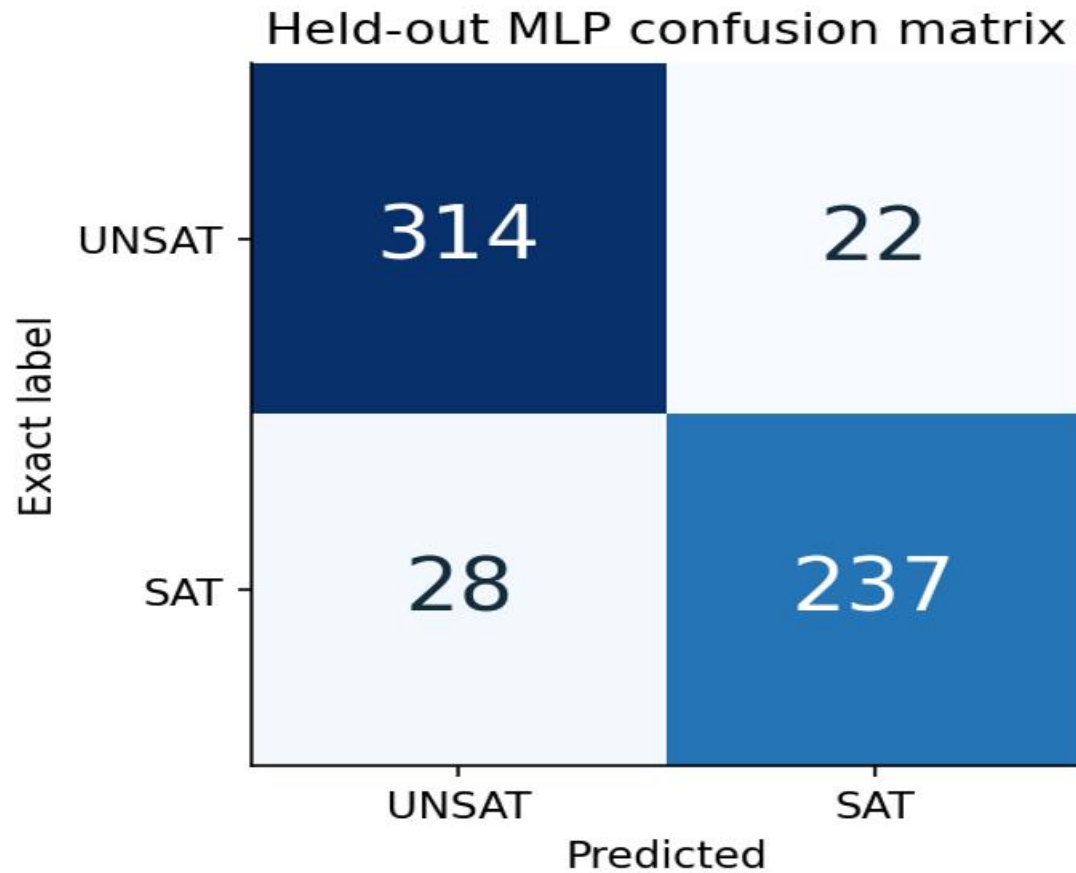
Adam lr = 0.001
Batch size = 64
Weight decay = 0.0001

Recorded run

Seed = 42; CPU
44 epochs executed
Best checkpoint: epoch 19
Early-stop patience = 25

Save weights, per-epoch losses, preprocessing and split group IDs.

Held-out results: useful, but still fallible



91.68%

MLP test accuracy · 551 / 601 correct

Linear baseline	90.85%
Majority baseline	55.91%
SAT precision	91.51%
SAT recall	89.43%

22 false SAT / 28 false UNSAT.
Feature-distinct subset: 91.62% (597).
One split; no significance claim.
Exact solving is still the verifier.

What was built, and what was learned

Reproducible deliverables

- Generator + immutable dataset
- Pandas cleaning + EDA
- PyTorch training + saved inference
- Interactive prediction / exact check
- Report, English slides and tests

AI assistance and limits

Codex helped draft and debug code, tests, plots and documentation.

Small synthetic distribution; lossy features; no solver replacement or quantum-advantage claim.

```
bash scripts/run_sat_app.sh
```

Demo: <http://127.0.0.1:8766>

A trained neural prediction is checked against exact classical behavior.